SYSTEM SPECIFICATION · PHASE 1

AI5K Profile Intelligence System

How the system collects a freelancer's evidence, measures it against the top tier, and rebuilds their profile toward $5,000 a month

Nine ingestion sources into one schema. Every claim tied to the place it came from. A benchmark of what actually wins in each niche. Seven weighted dimensions, ranked gaps, and generated content that cannot say anything the user is unable to prove.

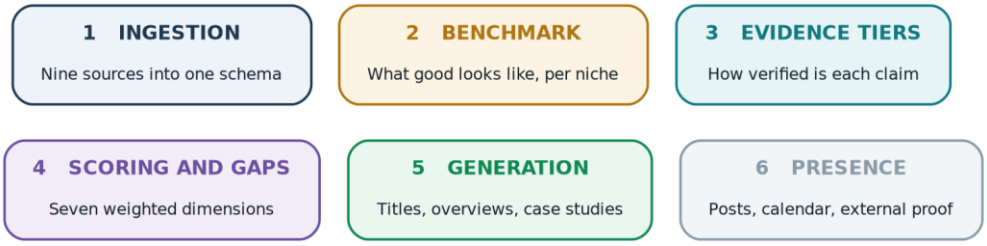
July 2026 · Build specification · Supersedes all earlier drafts

Contents

Right-click and choose Update field to populate page numbers.

1. System overview

The system collects a freelancer's data from every source where they have public evidence, classifies how well each claim is supported, scores the profile against what the top tier in their niche actually does, ranks the gaps by return on effort, and generates the improved profile. Nothing is scraped on a user's behalf from a platform that forbids it, and nothing is published that the user cannot prove.



Six components. Ingestion and benchmark feed scoring; scoring drives everything downstream.

The rule the whole system rests on

Every claim on a generated profile points back to the exact place it came from: a specific sentence, a specific commit, a specific line of an invoice.

A claim with no source span is not published. It becomes a coaching prompt instead, telling the user how to earn that proof.

This is what separates the system from keyword-matching tools, which raise a similarity score by inventing achievements. On Upwork an inflated claim survives until the first client conversation and then destroys the private feedback score that controls everything else.

The claim record

The unit of storage is a claim, not a profile field. Everything downstream reads from these.

ONE CLAIM RECORD

A claim with no source span is never publishable

claim_text	Built a RAG system over 100k documents
skill_ids	rag_systems, vector_db, langchain
source_type	cv_upload + github_repo
source_span	page 2 line 14 · repo rag-pipeline
evidence_tier	T2 project demonstrated
weight	0.85
observed_date	2026-03-11 (last commit)
recency_factor	1.0
publishable	true

Every claim carries its origin, its strength and its age

Two consequences for the build. Original files are retained, not just extracted text, because grounding means re-reading the source at generation time. And every claim carries a date, so a 2019 project is discounted against a 2026 one rather than counted as equal.

2. Data ingestion

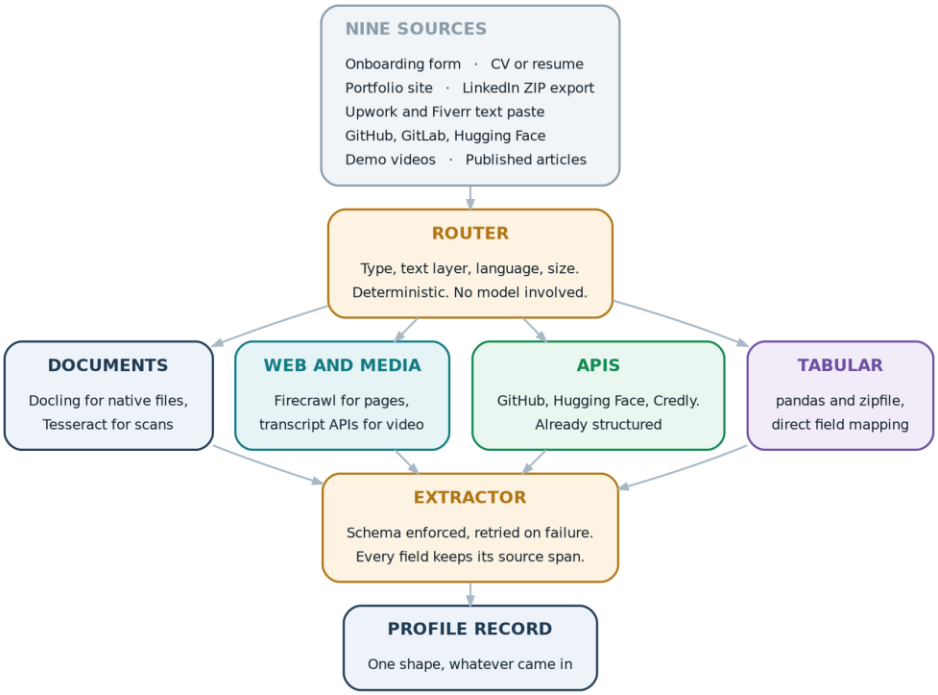
Nine sources, one output shape.

Source	How it arrives	Yields
Onboarding form	Structured answers no parser could find: target vertical, target earnings, hours available, biggest struggle	Context and goals
CV or resume	PDF or DOCX upload, layout-aware parse	T6 work history, T8 skills
Portfolio site	User supplies the URL, full crawl of their own asset	T2 projects
LinkedIn	Native ZIP export the user downloads and uploads	T6 history, T7 recommendations
Upwork and Fiverr	User selects their own profile text and pastes it in	T1 where reviews name outcomes
GitHub and GitLab	API pull: languages, topics, stars, commit recency	T2, the strongest technical proof
Hugging Face	API pull: published models and datasets, download counts	T2 with adoption signal
Demo videos	User supplies URLs, transcript extracted, tools and metrics pulled	T2 demonstration
Articles and threads	Crawled from URLs the user provides	T2 thought leadership

The LinkedIn export and the Upwork text paste both deserve emphasis. They deliver high-value data with no terms-of-service exposure at all, because the user performs the retrieval and the system only parses what they hand over. No scraper is used against any platform on a user's behalf.

Routing before parsing

Files are classified first and routed second. A native PDF and a photograph of a certificate need different treatment, and running a vision model across clean text is slow and expensive for no gain.



Classification is cheap and deterministic. Only the expensive parsers see documents that need them.

Mandatory pre-parse check

Sample a few pages of any PDF and read the extracted text. If it comes back empty or as noise, the file is a scan wearing a PDF extension and must be routed to OCR.

Without this check the profile fills with garbage, and the failure is silent.

Extraction

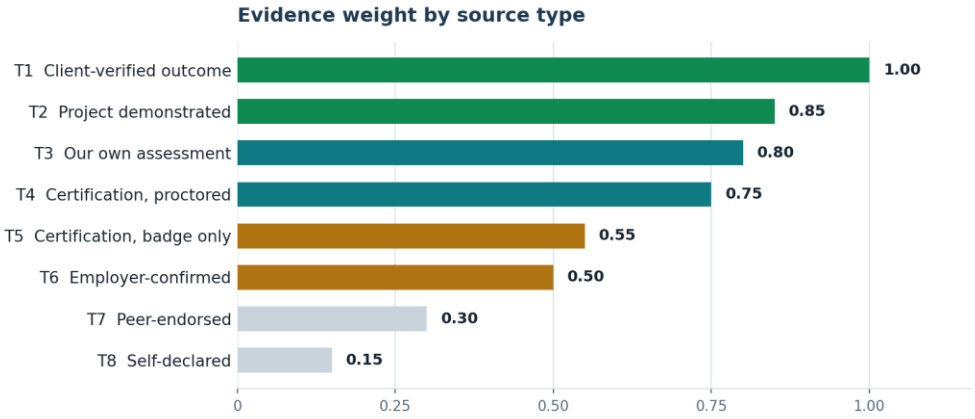
Extraction runs as several small parallel calls rather than one large one, with a separate specialised prompt for identity, for work history, and for skills and proof. Splitting the task improves accuracy and reduces latency at the same time, because a single prompt asking a model to identify many kinds of information at once performs worse at all of them.

Output is validated against a schema and retried on failure. Every field is nullable. Forcing a required field when the data does not exist is the most reliable way to make a model invent one, and a missing field is a true and useful answer that becomes a coaching prompt later.

The model returns indices into the source text rather than paraphrased content, and post-processing re-extracts the exact original block from those indices. This is what makes span grounding real rather than aspirational: the model never gets an opportunity to paraphrase a proof point into existence.

3. Evidence classification

Every claim is assigned a tier by how well it is corroborated. The tier determines its weight in scoring and whether it can appear on a public profile.



Eight tiers. A skill takes its strongest evidence, never the sum of weak evidence.

Tier	Definition	Weight
T1	Client-verified outcome, named in a review or testimonial	1.00
T2	Project demonstrated: deployed repository, live application, published model	0.85
T3	Platform-assessed: our own structured skills assessment	0.80
T4	Certification, proctored and verifiable with the issuer	0.75
T5	Certification, badge only or self-paced course	0.55
T6	Employer-confirmed: work history where the skill was used	0.50
T7	Peer-endorsed: recommendation from a colleague	0.30
T8	Self-declared, with no corroboration	0.15

Why certifications are split across two tiers

A proctored cloud architecture certification and a self-paced course certificate are not comparable evidence, and treating them as one tier either overvalues the weak ones or undervalues the strong ones. Splitting them at T4 and T5 resolves it. Where a certification cannot be verified with the issuer at all, it drops to T8.

Why shipped work outranks certification

An AI buyer on Upwork hires on evidence of building, not of studying. A repository with eight months of commit history is materially harder to fabricate than a certificate, and it demonstrates the thing the buyer is paying for. T2 therefore sits above T4.

The evidence cap

A profile with no proof cannot score well

When every claim on a profile is self-declared, the evidence dimension is capped at 3 out of 10, and the overall readiness score is capped at 30.

Without the second cap a user could reach the seventies on positioning, keywords and completeness while proving nothing, which is exactly the inflated profile the system exists to prevent.

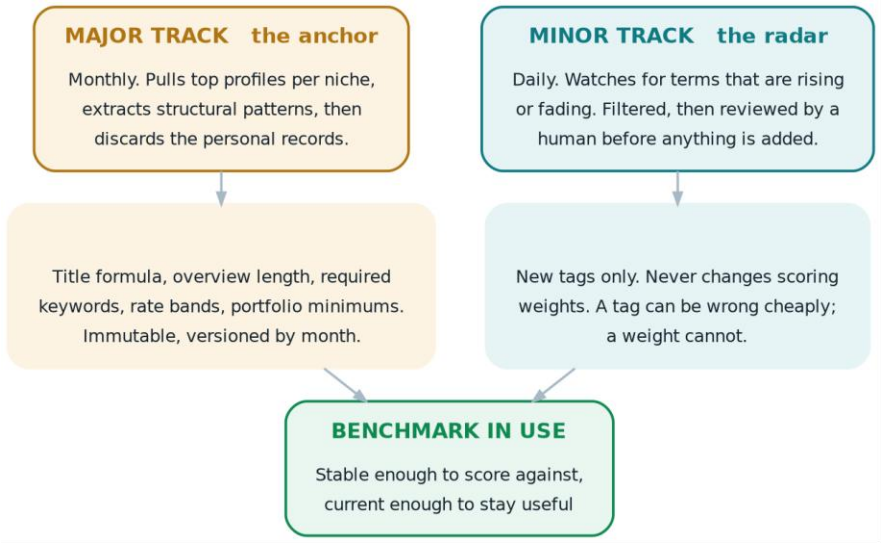
What a new user sees

Self-declared evidence is weighted at 0.15, which means a person who uploads only a CV starts with a low score. That is arithmetically correct and it is also the majority case for early-career users, so the presentation matters as much as the number.

The interface never leads with a bare figure like four out of a hundred. It leads with the position and the route out of it: you have eleven claims and can prove two, and here are the three fastest ways to prove three more this week. The arithmetic underneath is unchanged; the first experience is completely different.

4. Benchmark engine

The benchmark defines what a winning profile looks like in each niche. It runs on two tracks at different speeds, because the structural patterns change slowly while the vocabulary changes weekly.



A slow track to score against, a fast track to keep the language current

The anchor track

Monthly. Pull top-performing profiles per niche, extract the structural patterns, then discard the personal records.

Patterns are stored. Profiles are not.

What is kept: title construction, overview length, how many proof points appear, portfolio item counts, which terms recur, where rates cluster for a given evidence level.

What is discarded: the profiles themselves, immediately after extraction.

This is both the safer position on personal data and the more useful one, since the aggregate statistics are what the scoring consumes. Holding the underlying records adds risk and no capability.

The output is versioned by month and immutable once written, so a score computed in March can always be explained by the March benchmark.

On sample size: a hundred profiles per niche is the target for stable percentile statistics. Fifty split across three niches is too thin to trust. Expect to accumulate this across several refresh cycles rather than in one pass.

The radar track

Daily. Watch for terms that are rising and terms that are fading, filter them, and route them to a human before anything is added.

The radar produces tags only. It never alters scoring weights. A tag that turns out to be noise costs nothing to remove; a weight that turns out to be wrong silently distorts every user's score until someone notices.

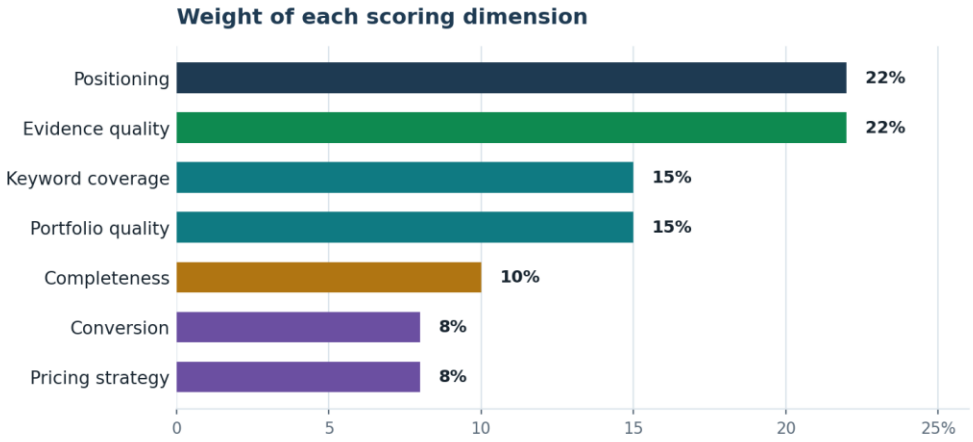
Watching for decline matters as much as watching for emergence. A profile written in last year's vocabulary reads as stale to a buyer, and nothing else in the system would ever detect that.

Benchmark record

```
niche               SMB workflow automation
title_formula       [role] - [vertical] - [outcome]
required_terms      n8n, Make.com, API integration, webhook
overview_words      280 - 420
portfolio_min       3 items, at least 2 with numbers
rate_band           $50 - $80 given T1 or T2 evidence
version             2026-07   (immutable)
```

5. Scoring

Seven dimensions produce a Profile Readiness Score from zero to one hundred.

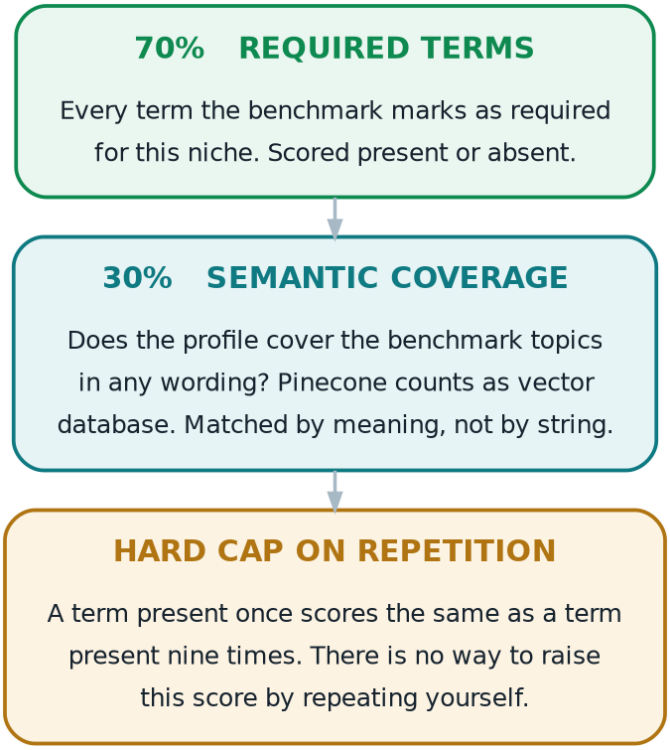


Positioning and evidence quality together account for 44 percent

Dimension	What it measures	Weight
Positioning	Role, vertical and outcome stated with enough specificity to be found and believed	22%
Evidence quality	Sum of tier weights across claimed skills, subject to the cap	22%
Keyword coverage	Required terms present, plus semantic coverage of benchmark topics	15%
Portfolio quality	Item count, quantified results, working links	15%
Completeness	Checklist of essential profile elements	10%
Conversion	Whether the opening addresses the buyer's problem rather than describing the seller	8%
Pricing strategy	Whether the stated rate is defensible given the evidence tier	8%

Keyword coverage

Keyword coverage is scored on presence, not frequency.



There is no way to raise this score by repeating yourself

Frequency-based similarity is deliberately not used. It rewards repetition, which builds a stuffing incentive directly into the metric users are being asked to optimise, and published analysis of keyword-matching tools shows that pushing match scores higher through repetition makes profiles read unnaturally without improving hire rates. It also cannot match meaning: a buyer searching for a vector database wants the person who wrote Pinecone, and bag-of-words will not connect the two.

Skill gaps are reported, not scored

Which skills the top tier holds that a user lacks is reported as a separate output rather than as a scoring dimension. It is a learning signal, not a profile-writing fix, and penalising someone for a skill they have not acquired yet would be both unfair and useless. Told plainly, it is often the most valuable thing the system says: here is what to go and build next.

6. Gap ranking

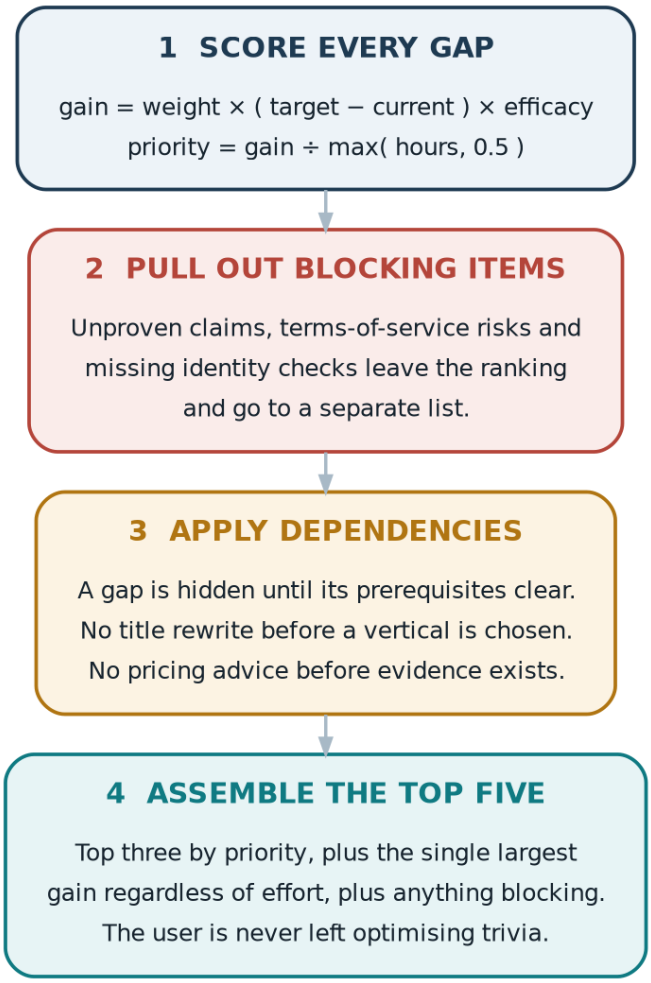
Each gap is ranked by how many score points it returns per hour of effort.

```
gain     = weight x (benchmark_target - current) x efficacy
priority = gain / max(effort_hours, 0.5)
```

Term	Source
weight	The dimension weight from section 5
benchmark_target	The value the top tier reaches in this niche, not a perfect 100
current	The user's current score on that dimension
efficacy	Expected share of the gap this fix actually closes, from a versioned lookup table of fix types
effort_hours	Estimated hours, floored at 0.5 so trivial fixes cannot dominate the ranking

The target is the benchmark rather than a perfect score. Telling someone to reach 100 on completeness when the top tier sits at 85 wastes their effort on the wrong thing.

The efficacy table is versioned and improved deliberately over time as outcome data accumulates. It is not re-estimated per run, because a figure guessed fresh each time is a guess wearing the clothes of arithmetic.



Ranking alone cannot express blocking, dependency or balance. Three rules handle those.

Blocking items leave the ranking

Unproven claims, terms-of-service risks and missing identity verification are not optimisation opportunities. They are liabilities, and they appear in a separate fix-before-publishing list regardless of what their return on effort would have been.

Dependencies gate what is shown

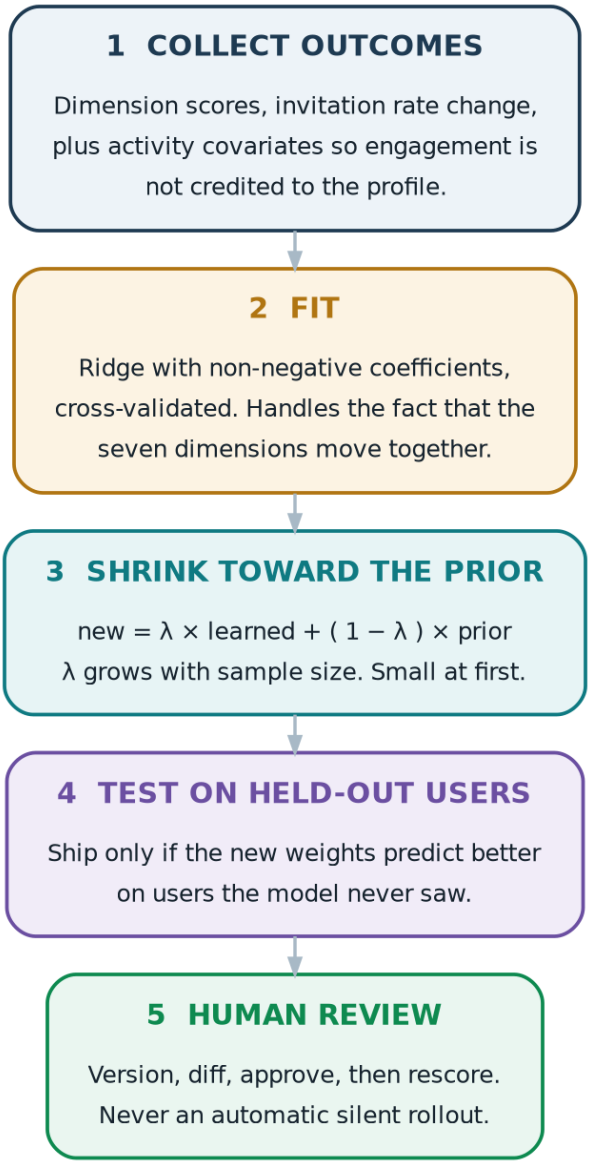
A gap stays hidden until its prerequisites clear. Rewriting a title around a vertical is meaningless before the vertical is chosen, and pricing advice is premature before there is evidence to justify a rate.

The top five stays balanced

The list is assembled as the top three by priority, plus the single largest available gain regardless of effort, plus anything blocking. Pure return-on-effort ranking would fill the list permanently with five-minute fixes and the user would never be told to build the portfolio piece that actually moves their earnings.

7. Keeping the system current

Dimension weights start hand-set and are refined from outcomes once enough users have reported results. The refit is deliberately conservative.



Five stages, with a human at the end of every one of them

Requirement	Reason
Coefficients keep their sign	A negative coefficient means the dimension is hurting outcomes. That is a finding worth investigating, not a magnitude to be absorbed. Taking absolute values would convert the discovery that a behaviour harms users into an instruction to do more of it.
Ridge with non-negativity, cross-validated	The seven dimensions move together, because a user who fixes positioning usually fixes keywords in the same session. Unregularised fitting on correlated predictors gives unstable coefficients that swing on small data changes.

Requirement	Reason
Shrink toward the hand-set prior	New weights are blended with the existing ones, with the learned share growing as the sample grows. At fifty users the learned share should be small.
Control for activity	Proposals sent and response time enter as covariates, so gains that came from engagement are not credited to the profile dimensions.
Test on held-out users	New weights ship only if they predict better on users the model never saw.
Minimum sample	Seven predictors need well over a hundred users before the fit is worth trusting, and that assumes clean outcome data.
Human review before rollout	Weights are versioned, diffed and approved. Automatic silent rescoring of every user is not permitted.

Where a specific question matters enough, an experiment beats a regression. Randomising which gap is surfaced first across users gives a causal answer about that gap, which no amount of observational fitting can provide.

Platform change monitoring

Upwork's ranking behaviour shifts, and community reverse-engineering of those shifts is tracked in a versioned configuration file. Changing a constant there triggers a background rescore for all users, and that change goes through the same review gate as a weight change.

8. Content generation

Every generated asset is bounded by the evidence store. A validator re-reads the source span before a claim is allowed through.

Asset	Structure	Hard constraint
Title	Role, vertical, measurable outcome	Every number traced to a stored source span
Overview	Hook on the buyer's problem, proof, process, call to action	Proof section drawn only from T1 to T4 claims
Case study	Problem, approach, result, evidence tier	Tier shown to the user, not hidden
Proposal draft	Job description matched to the strongest relevant evidence	Draft only. Never auto-submitted.

The constraint is enforced structurally rather than by instruction. A number that cannot be traced to a span does not reach the draft, so there is no opportunity for a plausible-sounding figure to survive review because it looked reasonable.

9. Presence engine

Enterprise buyers check a person's public footprint before hiring, so an empty external presence is itself a gap. The system generates drafts for LinkedIn, X and long-form publishing from the radar track and the user's own evidence, presents them for review, and syncs approved posts to a calendar with the full text in the event body.

Nothing is auto-posted. The user reviews and publishes.

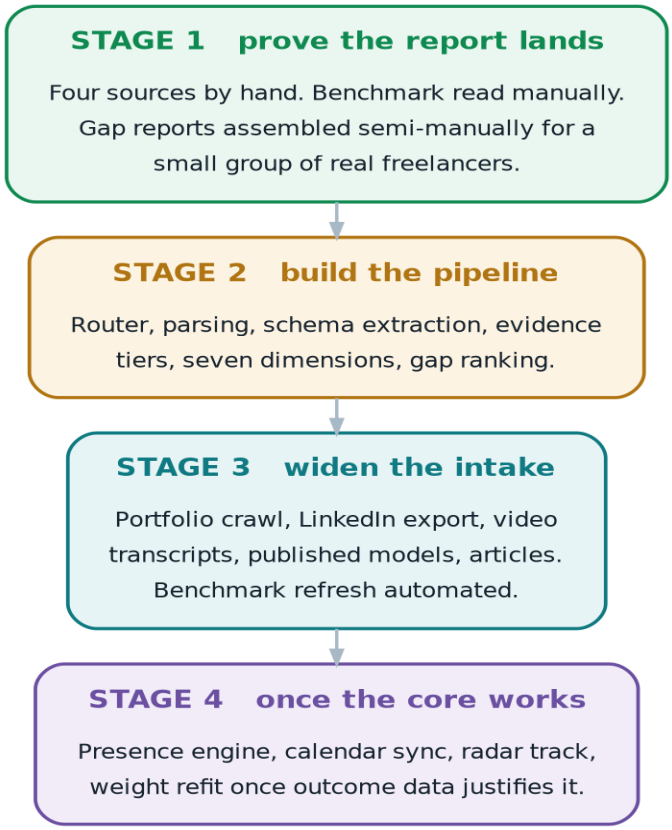
This ships after the core loop is working. It depends on the radar track, the evidence store and the generation layer all being in place, and starting it earlier would split the team across two products.

10. Tool stack

Job	Primary	Alternative
Onboarding form	Hosted form with webhook to the database	Form built into the application
Document parsing	Docling	LlamaParse, Unstructured
Scanned documents	Tesseract, managed OCR on failure	PaddleOCR for non-Latin scripts
Schema enforcement	Pydantic with a validating wrapper	Constrained decoding at higher volume
Field extraction	Small fast model per field group, parallel	One larger model if latency allows
Site and article crawl	Firecrawl	Jina Reader for single pages

Job	Primary	Alternative
LinkedIn	Native ZIP export parsed with pandas	None. No scrapers here.
Marketplace benchmark	Scheduled scraper actors, monthly	Managed data vendor
Code evidence	GitHub REST and GraphQL, Hugging Face Hub	GitLab API
Video transcripts	Transcript API, managed ASR as fallback	Self-hosted Whisper
Radar sources	Search API plus community sources	Manual review either way
Skill normalisation	NER model plus taxonomy mapping	Pre-built skills extraction library
Deduplication	String similarity, then embedding clustering	Probabilistic record linkage at scale
Embeddings	Self-hosted multilingual model	Managed embedding API
Generation	Strong general model	Second provider for redundancy
Database	PostgreSQL with vector support	Managed Postgres
Original files	Object storage, retained for span re-reading	Non-negotiable if grounding is real

11. Build order



The first stage needs almost no engineering, and it decides whether the rest is worth building

Parsing is commodity work and it keeps getting cheaper. The benchmark and the evidence model are the parts that are genuinely ours, and both can be tested by hand before any pipeline exists.

The question the first stage answers: does the gap report tell a working freelancer something they did not already know, and do they act on it? A small group of users, hand-assembled reports, a benchmark built by reading profiles manually. If the reports land, build the pipeline. If they do not, no amount of parsing infrastructure will rescue it.

What the user sees

PROFILE READINESS 41 / 100

Capped at 30 until claims are proven

DIMENSION	YOU NOW	BENCHMARK	PRIORITY
Positioning	Generic AI Developer	Role, vertical, outcome	**BLOCKING**
Evidence	2 of 11 claims proven	Top tier proves most	**BLOCKING**
Keywords	Missing 4 required terms	Buyers search these	**3.2 pts/hr**
Portfolio	1 item, no metrics	Three with numbers	**1.8 pts/hr**
Conversion	Opens with I am a	Opens with their problem	**1.5 pts/hr**
Pricing	$25/hr	Evidence supports $55-70	**0.9 pts/hr**

Blocking items first, then ranked by points per hour

There is deliberately no single headline percentage beyond the capped readiness score. A single match number pushes people toward keyword stuffing and away from telling a true story, and reporting per-dimension gaps with actions preserves exactly the information that makes the report useful.

12. Open decisions

Decision	The tension
Show benchmark numbers, or only the gaps	Showing them is more persuasive and invites gaming.
How hard to flag unproven claims	Too gentle and profiles stay inflated. Too harsh and the first interaction feels like an accusation.
Does identity verification gate generation or only paid activation	It moves conversion significantly either way.
How long original files are retained	Span grounding needs them, storage costs money, privacy law prefers deletion. A fixed window is likely the answer.
Whether skill-gap reporting suggests specific learning resources	Useful, but it turns a diagnostic product into a curriculum product.

The line that governs every other decision

If a claim cannot be traced to a source, it does not appear on the profile. Everything else in this specification exists to make that rule practical at scale.